Formalisme O93 – Architecture unifiée d'AAM orientée observation et acquisition de compétences

AAM – spécification normative d'architecture et de fonctionnement

12 octobre 2026

Résumé

O93 formalise un AAM où chaque fonction concourt à un cycle commun de résolution, de reproduction ou de création de mécanismes. La contribution structurante est un planificateur de compétences fondé sur l'observation et la génération adaptative de sondes : il réutilise les sondes existantes, compose leurs observations, applique les sondes miroir, diagnostique les collisions persistantes et propose de nouvelles sondes selon un gain mesurable. Les compétences sont des artefacts comportementaux évalués source-off, susceptibles d'être composés et adaptés sous conditions de domaine. Le RAG devient une famille de capacités optionnelles ; le mode par défaut observe les documents, construit des empreintes et apprend seulement lorsque les données soutiennent un objectif ou une expérience vérifiable. Une boucle autonome persistante orchestre le tout et garde une mémoire de raisonnement contrôlable. Les dispositifs et fichiers passent par une interface d'observation typée. Le présent document est la spécification cible ; la checklist associe chaque obligation à un gate et à des preuves. Une fonctionnalité décrite mais non testée conserve un statut non accepté.

Table des matières

1 Objectif, portée et principes normatifs **3**
- 1.1 Mission commune . . . 3
- 1.2 Portée réelle et règle d'honnêteté . . . 3
- 1.3 Axes de fonctionnement . . . 3

2 Architecture unifiée **4**
- 2.1 Couches et graphe de fonctionnement . . . 4
- 2.2 Contrat transversal . . . 4
- 2.3 Graphe de compétences . . . 5

3 Observation universelle **6**
- 3.1 Définition . . . 6
- 3.2 Sources supportées et adaptateurs . . . 6
- 3.3 Chargeur de capteurs et caméras . . . 6

4 Théorie des empreintes et moteur adaptatif de sondes **8**
- 4.1 Empreinte hiérarchique . . . 8
- 4.2 Ordre obligatoire du moteur de sondes . . . 8
- 4.3 Gain d'une sonde . . . 8
- 4.4 Sondes miroir et méta-observation . . . 9

5 Compétences de base, acquisition et composition **10**
- 5.1 Compétences de base exigées . . . 10
- 5.2 Acquisition par traces ou observation d'un mécanisme . . . 10
- 5.3 Pipeline d'acquisition . . . 10
- 5.4 Perfection et généralisation . . . 11

6 Substrat neuronal soumis aux empreintes **12**
- 6.1 Neurone-empreinte . . . 12
- 6.2 Objectif multi-termes . . . 12
- 6.3 Optimisation et mémoire distincte . . . 12

7 RAG et apprentissage à partir des documents **14**
- 7.1 RAG en tant que compétence optionnelle . . . 14
- 7.2 Mode normal : observer avant de rechercher . . . 14
- 7.3 Apprentissage global / Internet . . . 14

8 Orchestration et autonomie persistante **15**
- 8.1 Cycle de fond . . . 15
- 8.2 Arrêt, pause et mémoire . . . 15
- 8.3 Décomposition, outils et récompenses . . . 15

9 Conversation, génération, images, audio et vidéo **16**
9.1 Conversation unifiée . . . 16
9.2 Création de contenu . . . 16
9.3 Image, audio, vidéo . . . 16

10 Interface utilisateur Windows-first **17**
10.1 Écrans obligatoires . . . 17
10.2 Windows et déploiement . . . 17

11 Sécurité, confidentialité et fiabilité **18**

12 Évaluation, promotion et maintenance **19**
12.1 Statuts normatifs . . . 19
12.2 Protocole d'acceptation . . . 19
12.3 Cycle des phases . . . 19

13 Plan de construction O93 et gates d'acceptation **20**

14 Conséquences, limites opérationnelles et état **21**
14.1 Conséquences théoriques . . . 21
14.2 Conséquences conceptuelles . . . 21
14.3 Conséquences opérationnelles . . . 21
14.4 État de réalisation . . . 21

A Schéma canonique de tâche **22**

B Gabarit obligatoire du rapport **23**

Chapitre 1

Objectif, portée et principes normatifs

1.1 Mission commune

AAM O93 est un système local-first d'acquisition de compétences. Il doit être en mesure de (i) observer un artefact, un programme, un mécanisme ou un environnement ; (ii) créer et combiner des empreintes typées ; (iii) inventer et tester des sondes ; (iv) apprendre ou reconstruire une compétence depuis des observations, des traces ou un accès temporaire à une source ; (v) la reproduire sans dépendre de cette source ; (vi) composer et adapter des compétences à une tâche ; (vii) répondre, discuter, écrire, transformer, générer ou résoudre des problèmes ; (viii) vérifier les résultats, apprendre des retours, se souvenir et reprendre après interruption.

Toutes les fonctions convergent vers un même objet opérationnel : un *TaskEnvelope* représentant une demande, un objet ou une initiative autonome. Les modules ne forment plus des îlots qui se répondent seulement par l'utilisateur. Ils sont enregistrés dans un graphe commun avec entrées/sorties typées, préconditions, risques, coûts, preuves, domaine, mécanismes de vérification et règle de mémorisation.

1.2 Portée réelle et règle d'honnêteté

Cette spécification exprime la cible de construction, et non une affirmation de capacité déjà réalisée. Toute ligne de checklist doit être liée à une version, un test, un résultat et un statut. Une compétence candidate n'est promue qu'après évaluation indépendante. L'absence de ressource, de matériel, de corpus ou d'environnement mène à `NOT_RUN`, `ENVIRONMENT_ERROR` ou `INTERRUPTED_BUDGET`, jamais à un verdict sur le potentiel d'AAM. À l'inverse, l'existence d'une classe de code ou d'un test jouet réussi ne prouve pas une fonction générale complète.

1.3 Axes de fonctionnement

Local autonome Application, mémoire, observation des fichiers, compétences locales, entraînement léger, RAG et boucle de fond, sans réseau exigé.

Local étendu Ajout de matériel et capteurs explicitement autorisés, modèles préentraînés locaux, données et ressources accélératrices optionnelles.

Global étendu Connecteurs, grands entraînements distribués et ressources Internet gouvernés par permissions, provenance, licences et budgets. Ce mode augmente les moyens, sans remplacer le mode local ni faire confiance au réseau par défaut.

Chapitre 2

Architecture unifiée

2.1 Couches et graphe de fonctionnement

1. **Interface et bus d'entrée** : chat, requête, glisser-déposer, fichier/dossier, dataset, caméra, capteur et connecteur.
2. **Contrat de demande** : ID, objectif, résultat attendu, contexte, contraintes, confidentialité, budget, politique réseau, politique de mémoire, vérificateurs, provenance et échéance.
3. **Observateur universel** : inventorie le type, décode ce qui est connu, préserve l'original et produit des vues ainsi que les états de décodage. Les adaptateurs matériels exposent la même interface.
4. **Moteur de sondes** : portefeuille typé/versionné ; réutilisation ; composition ; miroirs ; génération de variantes ; proposition de primitives ; mesure du gain, du bruit et du coût.
5. **Graphe d'empreintes** : observations, objets, transformations, relations, états, unités, hash, contextes et collisions.
6. **Mémoire** : mémoire source, mémoire d'épisodes, mémoire de raisonnement contrôlable, poids de compétences, graphes de compétences et mémoire de connaissance indexée.
7. **Acquisition de compétences** : préparation des traces, partitions, sélection de probes/features, apprentissage, distillation si applicable, capsule, replay source-off, transfert, calibration et promotion/révocation.
8. **Routeur/orchestrateur** : sélectionne compétence, composeur, RAG lexical ou vectoriel, outil ou modèle de langage selon tâche, domaine, coûts et preuves disponibles.
9. **Réalisation** : classer, extraire, transformer, programmer via mécanisme autorisé, générer une réponse, créer un texte/média ou proposer la prochaine observation.
10. **Vérificateur** : tests indépendants, exactitude, invariants, citations, contract checking, sécurité, OOD et abstention.
11. **Récompense** : mesure progrès, validation, nouveauté utile, feedback utilisateur, erreurs et coût. Elle ne peut ni modifier les permissions ni se déclarer preuve elle-même.
12. **Boucle autonome persistante** : ouvre un run, inspecte l'état, crée l'agenda, choisit les actions permises, observe, apprend, journalise, se checkpoint, et reste active jusqu'à l'arrêt explicite.

2.2 Contrat transversal

Chaque nœud du graphe implémente ‘execute(input, context) -> TaskResult‘. Le résultat contient ‘status‘, sortie structurée, preuves, erreurs, source hashes, versions de sondes/compétences, coûts, pré/postconditions, confiance calibrée si possible et références de checkpoint. Une fonction qui n'utilise pas un champ doit le marquer `NOT_APPLICABLE` ; elle n'est pas autorisée à inventer une

valeur. Les événements s'enchaînent par `prev_hash` et `event_hash`. Toute promotion de poids ou de compétence utilise un remplacement atomique avec conservation de la version antérieure.

2.3 Graphe de compétences

Un ‘CompetenceNode‘ porte : ID et version ; contrat d'entrée/sortie ; domaine ; transformations invariantes supposées ; famille d'empreintes ; probes utilisées ; paramètres/poids ; provenance de l'apprentissage ; jeux train/val/test ; preuve d'inférence source-off ; critères de confiance et d'abstention ; coûts ; effets ; droits requis ; date ; statut ‘BASE‘, ‘CANDIDATE‘, `VALIDATED_BOUNDED`, ‘REVOKED‘ ou ‘UNRESOLVED‘. Une `CompetenceEdge` indique conversion de type, composition séquentielle/parallèle, précondition, postcondition, effet de bord, budget et vérificateur. Aucun simple score de similarité ne promeut une compétence.

Chapitre 3

Observation universelle

3.1 Définition

L'observation O93 n'est pas synonyme de recherche de texte ni d'un seul appel de hash. Elle est une transaction de lecture qui prend un objet et un contexte et retourne un ensemble de vues typées, états de décodage, empreintes, métadonnées, provenance, erreurs et liens vers l'artefact original.

$$O(x, c) = \{(p_i, v_i, s_i, m_i, e_i)\}_i = 1^n$$

où p_i est la sonde versionnée, v_i sa valeur valide ou absente, s_i son statut, m_i ses métadonnées et e_i son coût/erreur. La signature de contenu, le résumé sémantique (s'il existe) et les métadonnées d'acquisition sont distincts.

3.2 Sources supportées et adaptateurs

— Texte/JSON/JSONL/CSV/TSV : parse et statistiques ; validation des schémas ; positions des valeurs invalides.

— Code : parse AST, fonctions/classes/imports, graphe d'appels quand analysable ; ne pas exécuter pendant l'observation blanche. L'exécution de reproduction relève d'une sandbox séparée et d'un droit distinct.

— Image : hash d'octets, décodage, dimension, colorimétrie, vue globale, tuiles/régions, transformations miroir/inversion contrôlées, silhouettes et coordonnées spatiales.

— Audio : signature brute, durée, canaux, échantillonnage, énergie, fenêtres temporelles, spectre relatif, silences et transformations temporelles explicitement qualifiées. La transcription est une capacité externe/local ASR séparée.

— Vidéo : conteneur, durée, horodatages, frames, empreintes par frame/segment, mouvement estimé, audio synchronisé et pertes de frame. Un jeu d'images n'est pas déclaré vidéo complète.

— Capteurs/caméras : adaptateur explicite doté d'un identifiant matériel, permission, unité, fréquence, timestamp, calibration, état de flux et bouton d'arrêt. Aucun enregistrement implicite.

— Fichier inconnu : taille, hash, type détecté, entropie/statistiques d'octets et statut `NOT_DECODED` ; l'observateur ne le décrit pas sémantiquement sans décodeur.

3.3 Chargeur de capteurs et caméras

Tout adaptateur a l'interface ‘discover()‘, ‘open(authorization)‘, ‘read()‘, ‘close()‘, ‘health()‘ et retourne des `ObservationRecord`. Les flux sont lisibles avec backpressure, timestamps monotoniques et horodatage mural. La politique inclut consentement visible, sélection de source, durée, résolution, fréquence, enregistrement on/off, diagnostic et fermeture en cas d'erreur. Un adapter absent est

signalé, pas simulé comme capture réelle. Les mêmes contrats supportent de nouveaux instruments, caméras USB/IP, micro, fichiers, ports série ou capteurs spécialisés via plugin.

Chapitre 4

Théorie des empreintes et moteur adaptatif de sondes

4.1 Empreinte hiérarchique

Pour un objet x, le contexte c et un portefeuille $P = (p_i)$:

$$\Phi_P(x; c) = ((id_i, version_i, status_i, value_i, unit_i, provenance_i, cost_i))_i = 1^k.$$

Les coordonnées peuvent être numériques, catégorielles, relationnelles, temporelles ou absentes. La projection vectorielle pour un réseau ne supprime pas le registre typé complet. Chaque transformation T a un contrat de domaine et une relation d'invariance déclarée ; l'invariance est validée par expérience, non imposée à toutes les modalités.

4.2 Ordre obligatoire du moteur de sondes

1. Réutiliser les sondes connues, leurs versions et cache validé.
2. Composer en parallèle des sondes compatibles et séquentiellement lorsque les types de sortie/entrée sont compatibles.
3. Appliquer les sondes miroir et les relations vers des références identifiées.
4. Détecter les paires/régions encore confondues, avec une définition du domaine et de la distinction désirée.
5. Générer plusieurs sondes candidates paramétrées (normalisations, tailles de fenêtres, partitions spatiales/temps, ordres de combinaison, statistiques de forme/spectre, relations AST/JSON). Toute candidate est une hypothèse, pas une réponse.
6. Estimer le gain potentiel et le coût à partir d'un ensemble de développement ; figer la candidate avant validation indépendante.
7. Tester gain, stabilité, invariants cassés, bruit, coût et non-régression sur validation, puis le holdout de confirmation non utilisé pour choisir.
8. Conserver les sondes utiles ; rejeter ou mettre en quarantaine celles qui n'apportent rien ou dégradent les invariants. Continuer à proposer d'autres candidates selon budget et checkpoint.

4.3 Gain d'une sonde

Pour des paires d'objets avec cible indépendante z_ij, on utilise des métriques séparées : séparation utile S, collision résiduelle C, faux conflits F, stabilité R, coût K et taux d'erreur E. Un critère prototype peut être :

$$J_probe = \alpha S - \beta C - \gamma F + \delta R - \lambda K - \mu E.$$

Les poids d'objectif sont versionnés et choisis par protocole déclaré. Les contraintes dures (intégrité, permissions, type, validité) sont des portes booléennes, pas des pénalités compensables. Les sondes sans vérité de référence ou métrique proxy indépendante restent `UNRESOLVED` plutôt que d'être choisies sur l'impression de richesse de la signature.

Pour les sondes évaluées sur une tâche supervisée, le moteur calcule une similarité typée $sim(o_i, o_j) \in [0, 1]$ sur les sorties d'observation (les hashes seuls servent à la provenance), une dissimilarité $d_{ij} = 1 - sim(o_i, o_j)$, puis compare les paires de labels distincts et identiques. La mesure pairwise A est la probabilité empirique qu'une paire inter-cible ait une dissimilarité supérieure à une paire intra-cible, avec poids 1/2 pour une égalité. Le rang pairwise est pénalisé par coût/erreurs selon le protocole déclaré ; les taux de faux-découpage intra-cible et de collision inter-cible restent des diagnostics séparés. Un candidat peut être transmis à l'apprenant si le rang est favorable, mais ni A ni J_{probe} ne promeut une compétence : seul l'entraînement aval, le replay source-off, la validation indépendante et le transfert décident.

4.4 Sondes miroir et méta-observation

Une sonde miroir observe une transformation et la compare à une référence définie (inverse, permutée, relue, autre vue, autre instrument) ; elle ne présume pas que l'objet transformé conserve son sens. La méta-observation observe le processus de mesure lui-même : version, paramètres, données, sortie, erreurs, temps, cible/verifier et perte. Une passe méta doit générer une hypothèse falsifiable et mesurer sa valeur marginale face au contrôle. Les niveaux méta peuvent se poursuivre ; l'arrêt est lié au critère de gain et au budget transparent, jamais à une limite affirmée du potentiel.

Chapitre 5

Compétences de base, acquisition et composition

5.1 Compétences de base exigées

1. **Observer** : choisir l'adaptateur, lire avec contrat et produire les états de connaissance.
2. **Créer des empreintes** : appliquer les sondes, composer les vues, normaliser sans détruire la provenance.
3. **Créer des sondes** : proposer, estimer, exécuter et conserver des candidates dans l'ordre imposé.
4. **Orchestrer** : convertir une demande en graphe de sous-tâches, choisir des compétences, suivre les preuves et reprendre.
5. **Raisonnement logique** : représenter assertions, règles, contraintes, contre-exemples, dépendances et incertitude ; séparer la preuve de la conjecture.

Compétences primitives additionnelles : parsing/validation, mesure et vérification, gestion de mémoire/checkpoint, transformation de données, composition typée, recherche lexicale, gestion d'artefacts multimédias et génération de rapport. Une compétence de base est codée avec contrat et tests ; elle n'est pas automatiquement une compétence universelle pour tout objet possible.

5.2 Acquisition par traces ou observation d'un mécanisme

Sources admissibles : démonstrations d'entrées/sorties, répétitions contrôlées, oracle temporaire, code source analysé statiquement, traces de programme sandboxé, adaptateur, document technique ou observations physiques. Le moteur marque ce qui a réellement été vu : résultats, état interne, contexte, version, bruit et disponibilité de l'oracle. La source n'est retirée qu'après acquisition et test source-off, pas avant par hypothèse.

5.3 Pipeline d'acquisition

1. Établir le contrat de compétence et la matrice de tâches, dont cas négatifs.
2. Générer/choisir le premier portefeuille de sondes ; enregistrer la baseline.
3. Acquérir les observations et faire des splits par source, sujet, mécanisme ou temps pour éviter les fuites.
4. Construire un candidat par la méthode adaptée : régression/classification, réseau FPN, modèle de séquence, règle/programme synthétisé, composition ou distillation.
5. Optimiser avec objectif mesurable, validation seule pour le choix et checkpoints atomiques.

6. Tester les prédictions sur la validation ; diagnostiquer les collisions résiduelles ; générer des probes supplémentaires selon le protocole.
7. Quand l'objectif de reproduction est atteint sur la matrice de développement, effectuer un replay source-off et un test indépendant sur le holdout ; tester le transfert sur contexte nouveau.
8. Si les tests échouent, revenir à l'analyse/probes ou préserver `CANDIDATE` ; si les budgets sont atteints, checkpoint et continuer à la prochaine session sans conclure à une impossibilité.
9. Sauvegarder une capsule sans handle d'oracle, avec poids/règles/sondes/version, provenance et preuves. Le contenu brut n'est inclus que si la politique le permet.
10. Composer et réutiliser la compétence dans le solveur commun, surveiller le drift et révoquer/-retourner en arrière si les gates se dégradent.

5.4 Perfection et généralisation

La demande « reproduction parfaite » est respectée comme critère de recherche par domaine : zéro erreur sur une suite exhaustive ou préinscrite, avec états et cas frontières. Pour les domaines ouverts, rapporter les bornes observées, couverture, calibration et limites d'échantillonnage. La généralisation exige un domaine/test différent (variations, sources, environnements ou compositions) et ne peut pas être déduite du seul succès sur l'entraînement. Le système ne doit pas bloquer de nouvelles probes parce qu'un premier seuil est atteint si des collisions pertinentes subsistent.

Chapitre 6

Substrat neuronal soumis aux empreintes

6.1 Neurone-empreinte

Un neurone O93 est une unité paramétrée dans un graphe de poids, dont les entrées sont des coordonnées typées dérivées d'empreintes et dont les portes de disponibilité/fiabilité tiennent compte du statut de sonde. À la différence d'un neurone standard recevant un vecteur arbitraire, l'entrée conserve une liaison vers identité/versions des probes, familles de vues, unités, référence, modalité et masque de validité. Une forme de calcul est :

$$u_j = \sum_iW_ji\,(m_ir_i)x_i + b_j, \qquad h_j = \sigma(u_j),$$

où m_i est un masque de disponibilité et r_i une fiabilité calibrée indépendante. Une branche non disponible ne devient pas un zéro implicite : son masque est visible et le modèle peut abstentionner si la couverture est insuffisante.

Le réseau comporte des projections spécialisées par famille (globale, régionale, temporelle, relationnelle, lexicale, AST/tabulaire), partage des poids lorsqu'une correspondance de transformation est justifiée, et une porte de fusion entraînable. L'ajout d'une nouvelle famille de sondes requiert une migration explicite du schéma de poids ou un adaptateur entraîné. Les embeddings et poids d'un LM existant restent identifiés comme branche de séquence, pas supposés « gouvernés par les empreintes » simplement parce que l'entrée a été fingerprintée.

6.2 Objectif multi-termes

L'objectif est défini comme un tuple et des contraintes, pas toujours une somme indifférenciée :

$$\mathcal{L} = w_tL_task + w_rL_reconstruction + w_cL_contrast + w_iL_invariant + w_gL_generalization + w_sL_$$

Chaque terme possède : dataset et labels/preuves requis, calcul, masque d'applicabilité, unités, disponibilité, gradients, règle de validation et statut. Un terme sans données utiles est `NOT_APPLICABLE` ou 'UNRESOLVED' ; il ne prend pas artificiellement zéro qui améliorerait la somme. Les contraintes de sécurité et d'intégrité sont portes dures non compensables. L'apprentissage peut être alterné ou multi-tâche, avec gradient clipping, scheduler, régularisation, sélection de sondes et early stopping sur validation.

6.3 Optimisation et mémoire distincte

Les capsules poids contiennent uniquement les paramètres nécessaires, le schéma d'empreinte, tokeniseur si nécessaire, les probes et leurs versions, la provenance, les métriques, les calibrations et la preuve source-off. La mémoire épisodique contient épisodes/sources ; la mémoire de raisonnement

est un journal à conservation activable ; la mémoire de connaissance contient chunks/index ; les poids n'absorbent pas automatiquement ces données sans entraînement. L'utilisateur peut autoriser l'entraînement, réviser, exporter, verrouiller ou supprimer les données selon une politique versionnée.

Chapitre 7

RAG et apprentissage à partir des documents

7.1 RAG en tant que compétence optionnelle

La recherche lexicale (BM25-like) est un nœud dans le graphe, sélectionnable manuellement ou automatiquement. Elle est utile pour retrouver littéralement des passages ; elle n'est pas assimilée au retrieval sémantique. D'autres nœuds possibles comprennent la recherche vectorielle d'embeddings, la recherche par graphe de relations, les filtres structurés, les index temporels, la recherche d'image/audio et les combinaisons. Chaque méthode est sélectionnée selon évaluation de domaine, coût et provenance.

7.2 Mode normal : observer avant de rechercher

Lorsqu'un utilisateur charge une source RAG, l'application inspecte chaque fichier : hash, format, schéma, source, contenu décodable, anomalies et limites. Elle construit des empreintes par document/chunk et inscrit les relations dans le graphe. Puis elle évalue si le contenu contient des cibles d'apprentissage (paires input/target, questions-réponses explicites, relations validées, code et tests, labels ou feedback humain). Si oui, un pipeline d'acquisition de compétence est proposé/exécuté suivant l'autorisation ; sinon il conserve la mémoire de contenu et propose des hypothèses à vérifier. Aucune cible n'est inventée pour rendre un entraînement possible.

Le solveur observe ensuite la demande utilisateur, estime la nature de la tâche, sélectionne les compétences ayant des preuves et choisit les ressources. Une réponse factuelle provenant des documents renvoie ses citations précises ; une réponse non supportée s'abstient ou pose une question. Le routeur peut choisir zéro recherche, recherche lexicale, embeddings locaux, compétence documentaire ou modèle de langage selon mode et preuves.

7.3 Apprentissage global / Internet

Internet, fichiers distants, capteurs et modèles externes sont fournisseurs explicites. Les contenus externes portent URI, date, hash, licence/provenance si connue, et sont traités comme données non fiables. Les instructions intégrées aux documents ne changent ni le système ni les permissions. Mode offline conserve l'observation et les compétences locales. Le pré-entraînement ou l'entraînement grande échelle peut utiliser un modèle de base, GPU ou ressources externes configurés ; les modèles/poids/datasets doivent avoir licence et hash. La configuration locale n'est pas limitée par l'absence de réseau : elle rend visible la capacité non disponible dans le mode choisi et conserve le travail en attente.

Chapitre 8

Orchestration et autonomie persistante

8.1 Cycle de fond

Au lancement de l'application, le contrôleur autonome démarre sans action manuelle séparée. Il charge sa mémoire et ses poids, vérifie les journaux et checkpoints, inspecte son état et ses compétences, construit un agenda limité, et répète : état -> hypothèse -> sonde -> exécution autorisée -> vérification -> récompense/feedback -> mémorisation -> checkpoint. Si aucun dataset ni but utilisateur n'existe, l'objet de travail est l'auto-observation de l'état propre et de l'historique : inventaire des modules, compétences indisponibles, erreurs précédentes, état des checkpoints, suggestions d'expériences falsifiables. Il ne simule pas de données et ne prétend pas faire une réflexion sémantique s'il n'y a pas de modèle de langage ou de signal pour cela.

8.2 Arrêt, pause et mémoire

Il y a trois contrôles distincts : suspendre les actions, arrêter la boucle autonome et couper l'enregistrement de contenu de raisonnement. La commande d'arrêt d'autonomie entraîne la fermeture coordonnée de l'application/runtime après checkpoint. Désactiver la mémoire de raisonnement ne tue pas les tâches : les compteurs de fonctionnement et les journaux minimaux de sécurité/intégrité restent séparés, tandis qu'aucun nouveau contenu de raisonnement n'est persisté. La préférence est explicite, visible et enregistrée. Au redémarrage, l'application indique la dernière session et le choix de reprise.

8.3 Décomposition, outils et récompenses

Le décomposeur transforme le contrat de tâche en graphe avec dépendances, critères de réussite, méthodes candidates, probes et vérificateurs. La récompense comprend progrès validé, utilité notée par l'utilisateur, nouveauté utile, coût et pénalité pour erreur/non-vérification. Le vote utilisateur (« utile », « partiel », « incorrect ») est une observation de feedback, pas un label absolu qui remplace les preuves. Aucune récompense n'autorise shell, accès réseau, caméra ou modification de fichiers au-delà des scopes. Les objectifs qui restent sans preuve sont 'UNRESOLVED' avec prochaine expérience et checkpoint.

Chapitre 9

Conversation, génération, images, audio et vidéo

9.1 Conversation unifiée

Le chat est une interface à la boucle de tâches, pas un routeur autonome détaché. Pour chaque message : identifier le but et les objets attachés ; observer les fichiers et leurs contrats ; sélectionner probes/compétences/RAG/LM ; produire une sortie avec niveau de support ; vérifier le format et les références ; enregistrer la feedback ; garder une possibilité de reprise. Le modèle de langage est une compétence parmi les autres. Un LM compact entraîné localement peut prédire/générer des tokens, mais ne doit être choisi pour une réponse factuelle que si la qualité et le support sont suffisants. Les modèles de fondation sont des plugins avec tokenizer/poids/version/licence explicites.

9.2 Création de contenu

Génération d'histoire, texte ou programme est représentée par un contrat de créativité avec contraintes et vérifications adaptées. Le vérificateur distingue conformité du format, respect des contraintes de l'utilisateur, similarité à une source et exactitude factuelle. Une tâche de création sans vérité de référence ne reçoit pas artificiellement un score d'exactitude ; son évaluation combine checklist de contraintes, préférences et jugement humain traçable.

9.3 Image, audio, vidéo

Les protocoles maintiennent une empreinte globale et hiérarchique (régions, patchs, frames ou fenêtres temporelles) et des vues inversées/mirroirs avec relation explicitée. Pour les images : profondeur de vue, géométrie, textures et sémantique externe sont séparées. Pour audio : temps, spectre, segmentation voix/non-voix, transcription, confiance et contexte sont distincts. Pour vidéo : garder les timestamps, ordre des frames, mouvement et alignement audio. Une seule signature d'énergie/silhouette n'est jamais appelée compréhension sémantique complète. La compétence générative par modalité doit être entraînée/branchée séparément si demandée.

Chapitre 10

Interface utilisateur Windows-first

10.1 Écrans obligatoires

1. Chat/tâches : texte, pièces jointes, progression, annuler/reprendre, sources, résultat/vérification.
2. Observer : charger fichier/dossier, sélectionner modalité, découvrir capteurs/caméras, démarrer/arrêter, prévisualiser, afficher les probes et statuts.
3. Sources/Connaissances : ajouter RAG, mode observation-first par défaut, recherche classique à sélectionner, inspecter chunks/empreintes, labels et opportunités d'apprentissage, supprimer/-reconstruire.
4. Compétences : base, candidates, validées, source-off, cartes domaine/preuves/coûts, composer, exporter, révoquer.
5. Laboratoire/sondes : générer variantes, budgets/seeds/splits, collisions, résultats, comparer runs et revue humaine.
6. Autonomie/Mémoire : statut de la boucle, agenda, feedback, fréquence, mémoire raisonnement ON/OFF, pause/stop, checkpoints.
7. Données/entraînement : schémas, partitionnement, détection doublons/fuites, progression, pertes, reprise et rollback.
8. Audit/paramètres : journaux, hashes, modèle/poids, CPU/GPU/RAM, stockage, réseau, allowlist, licences, sauvegardes et isolation.

10.2 Windows et déploiement

Installer/exécuter sur Windows 10/11 sans chemins hardcodés, avec Python/GUI prérequis explicitement vérifiés. Tester nom de dossier avec accents/espaces, droit normal, CPU seul, absence de réseau, défaut de stockage, caméra refusée, pause/arrêt, fermeture brutale, restauration et désinstallation. La UI ne doit jamais signaler « terminé » quand le moteur retourne une exception. Les changements de schéma et de weights sont atomiques ; le lanceur renvoie le code d'échec à l'OS.

Chapitre 11

Sécurité, confidentialité et fiabilité

Tous les fichiers, données, résultats d'outils et contenus réseau sont des entrées non fiables. Les lecteurs limitent octets/durée/profondeur et refusent traversal/symlink selon politique. Le code observé est analysé sans exécution ; toute exécution ultérieure est sandboxée. Les capteurs demandent une permission visible et peuvent être arrêtés immédiatement. Les commandes externes et les actions à impact nécessitent des scopes explicitement configurés. Les clés/jetons ne sont jamais mis dans le RAG ni les logs. Les capsules sont vérifiées par hash/schema avant chargement. Les mises à jour gardent backup, version et rollback. Les erreurs ne sont pas silencieusement converties en valeurs plausibles.

Chapitre 12

Évaluation, promotion et maintenance

12.1 Statuts normatifs

`NOT_STARTED`, ‘BLOCKED‘, ‘RUNNING‘, `PASS_BOUNDED`, `FAIL_QUALITY_GATE`, `NOT_RUN`, `ENVIRONMENT_ERROR` `INVALID_INPUT`, `INTERRUPTED_BUDGET`, ‘UNRESOLVED‘, ‘REVOKED‘, ‘COMPLETE‘ (seulement si tous les critères de sortie ont une preuve). ‘COMPLETE‘ signifie la phase de checklist, dans le domaine de test indiqué, pas l’accomplissement du potentiel général d’AAM.

12.2 Protocole d’acceptation

Pour chaque compétence : test contractuel ; validation indépendante ; source-off après sauvegarde/redémarrage ; holdout congelé ; OOD et cas d’abstention ; fuite/duplicates ; calibration ; performance/coût ; safety ; plusieurs seeds/sources lorsque pertinent ; report complet et preuve de hash. Pour UI : smoke Linux n’est pas Windows natif. Pour capteur : simulation de test n’est pas flux réel de l’appareil. Pour Internet : mock HTTP n’est pas récupération réelle. Pour le modèle langage : cross-entropy ne prouve pas une conversation de qualité.

12.3 Cycle des phases

Chaque phase lit avant exécution le formalisme, la checklist de travail et les deux registres d’erreurs ; prend checkpoint ; utilise des données et hashes identifiés ; écrit ‘run.json‘, ‘events.jsonl‘, ‘errors.jsonl‘, résultats et checkpoint final. À l’échec, la phase est ‘BLOCKED‘, le diagnostic et correction deviennent un nouvel événement, puis la même phase est rejouée. La phase suivante n’est pas démarrée avant tous les critères de sortie de la phase actuelle. Toute modification de code, données, protocole ou critères réouvre les phases dépendantes et impose un nouveau run.

Chapitre 13

Plan de construction O93 et gates d'acceptation

Phase	Livrable central	Critères de sortie nécessaires	Dépendances
P0	contrats, bootstrap, journal, checkpoint, préflight	tests d'intégrité ; démarrage isolé ; statut de checklist propre ; source O92 préservée et hashée	aucune
P1	observateur fichiers/multimodalité/adaptateurs	matrice de fichiers, hashes, états de decode, pas d'altération ; erreurs attendues ; API plugin	P0
P2	moteur de génération de sondes	ordre probes respecté, composition typée, miroirs, collisions, gain/coût, arrêt/reprise	P1
P3	compétences de base et graphe de compétences	5 compétences primitives exécutables ; contrats et composition ; tests unitaires/bout-en-bout	P1-P2
P4	apprentissage adaptatif/source-off	acquisition depuis traces, sondes adaptatives, split sans fuite, export, restart, transfert	P2-P3
P5	connaissance/RAG observation-first	4 modes, observation du corpus, apprentissage conditionnel, citations, support	P1-P4
P6	autonomie/mémoire/récompenses	service toujours actif en mode app ; auto-observation à vide, feedback, memo toggle, arrêt/reprise/checkpoint	P0-P5
P7	chat/creation multimodale/UI Observer	tâches passent par orchestrateur ; chargement fichier/capteur autorisé ; vraie UI et stop ; automation de feedback	P1-P6
P8	sécurité/intégrité/résilience	tests adversariaux, erreurs, rollback, vie privée, permissions, corruption, resource stress	P0-P7
P9	évaluation capacité	expérience preregistrée, source-off, OOD, plusieurs seeds, rapport validé	P2-P8
P10	Windows natif/global-scale	acceptance Windows 10/11, ressources et connecteurs déclarés, licences, offline/online	P0-P9

Chapitre 14

Conséquences, limites opérationnelles et état

14.1 Conséquences théoriques

Le mécanisme central transforme la demande en hypothèses observables, et l'observation en signatures versionnées qui peuvent être réutilisées pour apprendre. Les nouvelles sondes ne deviennent utiles que par gain mesuré de distinction ; la meta-observation n'ajoute pas automatiquement de l'information. Les poids neuronaux représentent des paramètres appris sur les coordonnées des empreintes ; la mémoire d'exemples et l'index documentaire gardent leurs identités distinctes.

14.2 Conséquences conceptuelles

La compétence n'est ni un passage RAG retrouvé ni un score du modèle. Elle doit produire un comportement autonome de sa source, avec domaine, préconditions, preuve, composition et possibilité de révocation. L'orchestrateur peut choisir la recherche lexicale, mais aussi l'observation d'un corpus, un entraînement local, une compétence spécialisée, le LM, un outil ou une combinaison. La capacité à créer une nouvelle sonde est une compétence de base méta ; ses propositions restent testées et falsifiables.

14.3 Conséquences opérationnelles

La boucle autonome est lancée au démarrage de l'application et ne s'arrête qu'à l'arrêt du runtime ou à l'action d'arrêt explicitement prévue. La journalisation de raisonnement est indépendante et peut être coupée ; la traçabilité minimale garde les événements de contrôle, d'intégrité et les checkpoints. Les périphériques et Internet restent des fournisseurs autorisés explicitement. Le mode local continue sans elles.

14.4 État de réalisation

Ce document est la spécification normative. Les phases d'implémentation sont suivies exclusivement dans `checklist_de_travail_093.json`. Une phase sera déclarée 'COMPLETE' uniquement après preuve de chaque critère, logs enregistrés, erreurs examinées et documentation mise à jour. Les critères non testables dans l'environnement courant restent ouverts et ne sont pas déclarés terminés par défaut.

Annexe A

Schéma canonique de tâche

```
TaskEnvelope = {
  "task_id": str, "goal": str, "mode": "auto|observe|learn|compose|chat",
  "inputs": [ObservationInput], "expected_output": Schema,
  "constraints": [Constraint], "permissions": [Permission],
  "budget": {"seconds": float, "steps": int, "bytes": int},
  "memory_policy": {"record_reasoning": bool},
  "retrieval_policy": "observe_first|lexical|vector|off|auto",
  "competence_candidates": [CompetenceID],
  "verification_plan": [ProbeSpec], "provenance": [ArtifactHash]
}
TaskResult = {
  "status": ResultStatus, "output": object, "evidence": [Evidence],
  "competences_used": [CompetenceVersion], "probes_used": [ProbeVersion],
  "metrics": object, "errors": [ErrorRecord], "checkpoint": str
}
```

Annexe B

Gabarit obligatoire du rapport

Description ; architecture ; pipeline de bout en bout ; commandes exactes ; environnement, datasets, graines et SHA-256 ; résultats bruts ; interprétation ; conséquences théoriques, conceptuelles et opérationnelles ; erreurs/limites ; conclusion ; statut ; prochaine phase. Le rapport indique clairement ce qui a été fait, ce qui a échoué, ce qui n'a pas été exécuté et la preuve de chaque conclusion.